

TaskFlow

TEST PLAN DOCUMENTATION

**Pericles Lelos, Krish Mohan
Sam Fickle, Charles Timmons**

Introduction

This test plan outlines the testing process for TaskFlow, a web-based task-tracking application made to assist users in tracking and organizing their assignments, chores, or responsibilities through an easy to use interface. The purpose of this document is to ensure that all primary features are thoroughly tested for usability and reliability. The testing will focus on the app's key functionalities, including but not limited to user authentication and authorization, task creation and edit, task status updates, and priority calculation. This test plan will focus primarily on a black box testing approach, and making sure that the user's interface works as necessary and provides a user-friendly experience.

Items Tested

- Ability to create a new todo card item
- Ability to edit and save pre-existing todo card items
- Priority correctly updates based on due date and time required
- Task card status correctly updates from completed/overdue
- Ability for a user to login via username and password
- Ability for a user to signup with a username and password

Items Not Tested

- Max amount of users in database does not affect site performance
- Max bound of task cards does not affect site performance

Approach

The overall testing approach follows a manual systems testing process. The majority of testing will be from a black box perspective, testing the frontend functionality rather than the code itself. Testing will include both positive and negative tests, ensuring that correct procedures/values work, and incorrect procedures/values do not. Testing will be constant throughout the development process, using an integration testing approach, and testing each new feature as it is implemented and added to the program.

Test Fail/Succeed Criteria

Tests focus on whether or not the user will have difficulty with a core feature. Tests will succeed if a feature is deemed intuitive, works as expected, and easy to use. Tests will fail if the feature gives the user difficulty or requires the user to have previous training or knowledge about said feature. Features that fail testing will be reported to the primary developers to undergo revision.

Test Case Examples

Test Case 1: User Sign Up

Use Case Reference: Use Case 1 – Sign Up for an Account

Purpose: Verify that a new user can successfully create an account.

Preconditions:

- User is not logged in.
- User does not already have an account with the same username.

Test Steps:

1. Navigate to the sign-up page.
2. Enter a valid username.
3. Enter a valid password.
4. Click "Sign Up."

Expected Results:

- System validates the inputs.
- A new user account is created in the database.
- User is logged in automatically or redirected to the login page.

Negative Tests:

- Username already exists → Error message displayed.
- Missing username or password → Input validation message.

Test Case 2: User Login

Use Case Reference: Use Case 2 – Log In

Purpose: Verify user authentication works properly.

Preconditions:

- User previously created an account.

Test Steps:

1. Navigate to the login page.
2. Enter valid username and password.
3. Click “Log In.”

Expected Results:

- System authenticates login credentials.
- User is routed to the home page displaying their tasks.

Negative Tests:

- Incorrect password → Error message.
- Non-existent username → Error message.

Test Case 3: Create a Task

Use Case Reference: Use Case 3 – Create a Task

Purpose: Verify that users can create new tasks and the system assigns priority.

Preconditions:

- User is logged in.

Test Steps:

1. Select "Create Task."
2. Enter task name, description, and due date.
3. Submit the form.

Expected Results:

- Task is stored in the database.
- Priority is automatically calculated.
- Task appears in the user's task list.

Negative Tests:

- Missing required fields → Error message.
- Invalid date format → Validation error.
- Database error → Task not saved message.

Test Case 4: Edit a Task

Use Case Reference: Use Case 4 – Edit a Task

Purpose: Verify that users can modify existing task details.

Preconditions:

- User is logged in.
- At least one task exists.

Test Steps:

1. Select an existing task.
2. Click "Edit Task."
3. Change task name, date, or description.
4. Save changes.

Expected Results:

- Updated task details appear in the task list.
- Database is updated.

Negative Tests:

- Invalid input (blank name, invalid date) → Validation error.

- Task not found → Display error message.

Test Case 5: Delete a Task

Use Case Reference: Use Case 5 – Delete a Task

Purpose: Verify that users can delete existing tasks.

Preconditions:

- User is logged in.
- Task to be deleted exists.

Test Steps:

1. Select a task.
2. Click “Delete Task.”
3. Confirm deletion.

Expected Results:

- Task is removed from the database.
- Task disappears from the task list.

Negative Tests:

- Task not found → Error message.
- Database error → Deletion failed message.

Test Case 6: Log Out

Use Case Reference: Use Case 7 – Log Out

Purpose: Verify a user can log out successfully.

Preconditions:

- User is logged in.

Test Steps:

1. Click “Log Out.”

Expected Results:

- User session is terminated.
- User is redirected to the login screen.